

Proof-of-Concept or Practical Tool? An Independent Reproduction of *AutoRecLab*

Robin Nitschke

Robin.Nitschke@student.uni-siegen.de
University of Siegen
Siegen, Germany

Simon Kontaxakis

Simon.Kontaxakis@student.uni-siegen.de
University of Siegen
Siegen, Germany

ABSTRACT

Recommender-systems (RecSys) research is increasingly bottlenecked by manual experimentation. A recent proof-of-concept, *AutoRecLab*, proposes to automate the engineering of RecSys experiments with a large-language-model (LLM) agent driven by tree search. We independently evaluate how reliably *AutoRecLab* works and how it can be improved. To this end, we re-ran its random-seed case study from the original natural-language prompt on a consumer-grade Windows PC. Across seven runs, the system was not runnable out of the box: we identified and fixed five platform- and framework-level defects, three of them Windows-specific. Output quality also appeared to track the backend model: gpt-5.4 produced valid LensKit 0.14.4 code on the first draft, whereas gpt-5.4-mini hallucinated the LensKit API in 6 of 13 debugging iterations of our single run with it. With one run per model this is an observation rather than a controlled comparison. With our patches applied, two end-to-end runs, the second a repetition of the first under identical settings, each produced complete metric outputs for all 9 sub-experiments (3 algorithms $\times$ 3 datasets, 5 seeds each) in one to three hours for about \$0.5–1 of API cost per run. Inspecting the generated code, however, shows that only 6 of these 9 are methodologically valid: in both runs the agent configured ItemKNN for explicit feedback, so that on unary data every recommendation receives the identical score. We confirmed this for the Run 5 node by re-executing its code, which yields exactly one distinct score per ItemKNN run, and for Run 7 by inspecting its constructor. This is the same misconfiguration the original study reports as its dominant failure mode, and it likewise escaped the agent’s reviewer. The consequence is that the ALS > Pop > ItemKNN ranking is an artefact of the configuration rather than a finding: under the implicit-feedback configuration documented for LensKit 0.14.4, ItemKNN’s nDCG@10 rises by a factor of 5.8 on MovieLens 100K and 24.8 on Last.FM across five seeds, and by 41.4 on the one Amazon split we re-measured, and it overtakes Most-Popular on all three datasets. What does reproduce is the original’s seed-sensitivity result, with coefficients of variation of 1.0–4.8% for ALS across the two runs. We therefore classify our result as a *partial qualitative reproduction*. On a deliberately small task, a minimal prompt finished in 7 minutes for \$0.30 at a score of 83.33/100. We conclude that *AutoRecLab* reliably produces complete and internally consistent metric outputs for small, clearly scoped tasks, but that nothing downstream of its reviewer stops a methodologically invalid configuration from being reported as a result. Platform fragility and LLM hallucination do not yet permit reliable paper-scale experiments, which is unsurprising for an early Alpha release. We reported a sixth defect upstream, where it has since been fixed; our five patches are available in our fork.

KEYWORDS

Autonomous Research Agents, *AutoRecLab*, Recommender Systems, Reproducibility, Large Language Models, LensKit

1 INTRODUCTION

1.1 Background

Autonomous research agents, often framed as “AI Scientists”, aim to automate parts of the scientific process that have traditionally required human researchers, from idea generation to experimentation and manuscript writing. Recent systems span the full spectrum, from end-to-end paper generators such as Sakana’s AI Scientist [8] and its tree-search successor [11], to autonomous machine-learning engineers such as AIDE that draft, debug, and benchmark code through tree search [6], to multi-stage research pipelines such as AutoResearchClaw that run the whole workflow from literature search to peer review [7]. While such systems have advanced rapidly in general machine learning, the recommender-systems community has largely overlooked automating its own research process, and existing tools are narrow, addressing only algorithm selection and hyper-parameter tuning [4].

1.2 Research Problem

Recommender-systems research is bottlenecked by manual experimentation. A single study couples data preprocessing, algorithm implementation, hyper-parameter choices, and statistical analysis, and the original *AutoRecLab* authors report that running their seed-sensitivity study by hand took hours to days of expert effort [4]. Agentic tools like *AutoRecLab* promise to remove this bottleneck by turning a natural-language description directly into a runnable, validated experiment.

That promise only holds if such a tool works for someone other than its creators. In the original proof-of-concept, 3 of 4 runs (75%) completed and 15 of 27 sub-experiments (56%) produced valid results, with the failures concentrated in identifiable methodological misconfigurations rather than random noise [4]. Because the system has so far been demonstrated only by its own authors, it remains unknown how dependably an independent user can run it on different hardware: whether it installs and executes out of the box, whether its findings reproduce, and what drives its sub-experiments to fail.

1.3 Original Work

AutoRecLab [4] is an LLM-based proof-of-concept, at the time of our study available as an early Alpha release, that automates the *engineering* of RecSys experiments. It does not yet generate research ideas, review literature, or write manuscripts; instead it translates a natural-language task description into an explicit specification

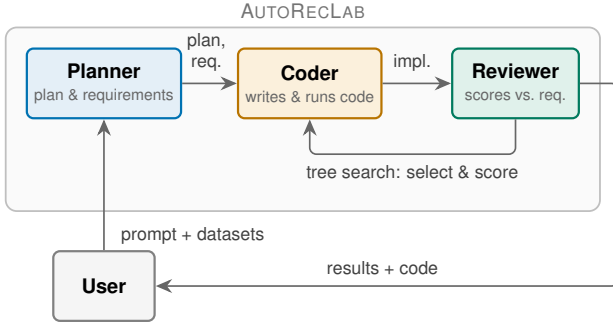


Figure 1: Architecture of *AutoRecLab*. A single LLM acts as **Planner**, **Coder**, and **Reviewer**; a tree-search controller selects which implementation to expand next. The user supplies a natural-language prompt and datasets, and receives a result overview plus the generated code. Redrawn after Beel et al. [4].

(task, dataset, splitting protocol, baselines, and metrics), generates and executes a LensKit [5] pipeline through iterative “run-repair” cycles, and validates the implementation against requirements that it derives from the prompt. This requirements-based validation is what sets *AutoRecLab* apart from general-purpose science agents: an implementation is judged on whether it meets explicit methodological conditions, not only on its final accuracy numbers.

AutoRecLab drives this process with a single LLM that acts in three roles, implemented through role-specific prompting (Figure 1). The *Planner* turns the prompt into an experimental plan and a list of binary (yes/no) requirements, such as correct dataset loading, an explicit train/test split, avoidance of data leakage, fixed metric definitions (for example $nDCG@k$), and reproducible execution with explicit random seeds. The *Coder* writes Python code, executes it locally, and debugs it from the captured stack trace when execution fails. The *Reviewer* checks the produced outputs against the requirements and assigns a score equal to the percentage of requirements that are met.

Each generation attempt is stored as a node holding the code, its execution output (including any errors), and the reviewer feedback; an implementation that fails to execute is flagged as buggy and scored 0%. A tree-search controller manages refinement across iterations: it usually expands the highest-scoring node, but with probability ϵ it explores an alternative branch to escape local optima. When it selects a buggy node, the *Coder* enters a debugging mode that repairs the error without changing the experimental design. The loop continues until an implementation satisfies all requirements or an iteration limit is reached; the system calls the OpenAI API and can be configured with different models.

The authors demonstrate the prototype on a case study of random-seed effects in RecSys evaluation, with three algorithms (ALS, ItemKNN, and Most-Popular), three datasets (MovieLens 100K, Amazon Video Games, and Last.FM), and the metrics $nDCG@k$ and $Precision@k$. They repeated the experiment four times with gpt-5 as the backbone and completed three runs of 10 or 20 tree-search iterations each, at \$1–5 and 2–9.5 hours per run. In these

runs no node ever satisfied *all* requirements, so the agent reported its highest-scoring node instead; the authors attribute this to the complexity of their nested requirements.

The authors report three main findings. Where outputs were valid, the agent’s results closely matched a human baseline (their own earlier, independently replicated [4] study of seed effects [10]) and supported the same conclusions. The generated code was, on inspection, functionally correct with only minor stylistic issues and was not hallucinated. The dominant failure mode was methodological rather than random: 15 of 27 sub-experiments (56%) were valid, and the 12 invalid ones were exactly the nine ItemKNN sub-experiments plus three Most-Popular ones, because ItemKNN was misconfigured for implicit feedback in a way that produced no error and therefore escaped the tree search. The authors position the system as a first proof-of-concept, explicitly noting that it covers only the engineering of experiments and not literature review, ideation, or manuscript writing.

1.4 Research Goal

Our goal is to independently establish whether, and how consistently, *AutoRecLab* reproduces its own proof-of-concept when run by an independent user on a consumer Windows PC, and to identify small, concrete improvements that make it more dependable. This work is done as part of the Machine Learning Lab [2].

2 METHODOLOGY

Reproduction target and prompts. We reproduced the random-seed case study of the original paper. To stay as close as possible to the original work, we re-used its natural-language prompt almost verbatim, asking the agent to quantify how data-split random seeds affect RecSys accuracy, using ALS, ItemKNN, and Most-Popular on MovieLens 100K, Amazon Video Games, and Last.FM with implicit feedback, 5-core filtering, five random seeds, user-based 80/20 hold-out splits, and $nDCG@k$ and $Precision@k$ for $k \in \{1, 5, 10\}$. After observing repeated execution timeouts (see below), we added a single clause asking the agent to use batch recommendation instead of per-user loops. The full paper-scale prompt, including this added final clause, reads:

“I’d like to run an experiment to quantify how much data split random seeds affect recommender system accuracy. Please use LensKit 0.14.4 to test three algorithms: ALS, ItemKNN, and Pop. Run this on the following three datasets with implicit feedback: MovieLens100K, Amazon Video Games, Last.FM. The raw files are stored in your working directory with the filenames u.data, VideoGames.csv, UserTaggedArtists-timestamps.dat. First, preprocess all datasets with 5-core filtering. For the Amazon and MovieLens datasets, please also convert any ratings greater than 3 to implicit interactions. Here’s the main experimental procedure: Generate 5 different random seeds for data splitting. For each algorithm, dataset, and seed, please do a user-based 80/20 holdout split. Train all models using standard hyperparameters. For the analysis, I need you to measure $nDCG@k$ and $Precision@k$ for $k=1, 5, 10$ and conduct a short statistical analysis. Please use batch recommendation for all users at once instead of per-user loops for efficiency.”

To test the system on a deliberately easier task, we also ran a reduced prompt with a single algorithm (ALS), a single dataset (MovieLens 100K), three seeds, and default hyper-parameters:

“Train ImplicitMF (ALS) on MovieLens100K (u.data in the working directory) using the default LensKit 0.14.4 hyperparameters. Apply 5-core filtering and treat ratings > 3 as implicit interactions. Use 3 different random seeds for 80/20 user-based holdout splits. Use batch.recommend for all users at once. Report mean and standard deviation of nDCG@10 and Precision@10 across seeds. Save results as a CSV file.”

System, model, and configuration. We ran the initial *Alpha* release of *AutoRecLab* [3], not the actively developed *develop* branch. All of our code, patches, prompts, logs, and results are available in our fork [9]; Section 6 states the exact repository state and what is archived where. The system builds on LensKit 0.14.4 and the OpenAI API. We ran the tree search with its default configuration (3 draft nodes, at most 10 improvement iterations, debug probability 0.3, exploration rate $\epsilon = 0.3$, and a per-execution timeout of 3600 s). We selected gpt-5.4 as the LLM backend and additionally tested gpt-5.4-mini to measure sensitivity to model size.

Datasets. We evaluated the three datasets of the original study: MovieLens 100K, the Amazon Video Games review data, and the Last.FM HetRec 2011 dataset. For MovieLens and Amazon we treated ratings greater than 3 as implicit interactions; for Last.FM we collapsed the user–artist tag assignments into interaction pairs. All datasets were 5-core filtered.

Hardware and cost. All experiments ran on a consumer-grade PC (AMD Ryzen 7 9800X3D, 8 cores / 16 threads, up to 5.2 GHz; 32 GB DDR5-6000 RAM) under Windows 11 with Python 3.11. We deliberately bypassed the project’s recommended Docker setup in order to test out-of-the-box usability in a native Windows environment. We report wall-clock runtime and estimated API cost per run; we did not measure energy consumption or CO₂.

Errors encountered. The original code could not run an experiment to completion on Windows. We encountered, diagnosed, and fixed five distinct defects; we summarise them in Section 3.1 (Table 1) because their discovery is itself one of our results. Because the system can only be evaluated once these blockers are removed, all quantitative results below were obtained with our patches applied.

Which node produced the reported numbers. During tree search *AutoRecLab* stores many nodes, and a later node can fail after an earlier one has already produced complete results. Naming a run is therefore not sufficient to identify a result. The values we report for Run 5 originate from the node generated in *tree-search iteration 9*, which received a reviewer score of 72/100 and was nevertheless marked unsatisfactory; it is the only node of Run 5 that executed successfully, so the agent fell back to reporting its best node. For Run 7 the corresponding node is *iteration 2*, node ID dac1170c438449f5a248475720dd3267, also scored 72/100 and also marked unsatisfactory. We archive both nodes with their complete logs, prompts, configurations, and result files under `artifacts/run5-selected-node/` and `artifacts/run7-selected-node/`.

Two points are worth stating explicitly, because they cost us time to establish. First, the file left behind in the working directory at the

end of a run (`runfile.py`) is *not* the evaluated node in either run; it is a later node that failed. We verified the identity of the evaluated nodes by hashing: the archived Run 5 code is byte-identical to the code embedded in the run log, and the archived Run 7 code is byte-identical to the code attribute of node dac1170c. . . in the tree-search state file. Second, node identifiers survive only for Run 7. *AutoRecLab* writes its tree-search state to a fixed path, so Run 5’s state file was overwritten by the later run; Run 5’s node is identified by iteration, reviewer score, log position, and code hash instead. Preserving per-run state would make provenance considerably easier to establish, and we note it as a further improvement to the tooling.

Metrics and analysis. For each (algorithm, dataset, seed) combination we report the mean and standard deviation of nDCG@10 and Precision@10 across the five seeds, together with the coefficient of variation ($CV = \text{std}/\text{mean}$) as a measure of seed sensitivity. We additionally report paired *t*-tests between algorithms on the per-seed scores; with five seed-matched pairs these are exploratory (Section 4). We compare every result against the baseline reported by Beel et al. [4].

3 RESULTS & DISCUSSION

We report five results. First, the system is not runnable on Windows without fixes, and we provide them. Second, output quality appears to depend on the LLM backend, though we ran each model only once. Third, with our patches the agent produced complete metric outputs for every algorithm–dataset combination, and the original study’s seed-sensitivity finding reproduces. Fourth, three of those nine combinations are nevertheless invalid, because the agent reproduced the original study’s ItemKNN misconfiguration; this removes the algorithm-ranking finding. Fifth, on a small, clearly scoped task the agent was fast and cheap, and inspection of its node found no methodological defect. We close by revisiting the role of the reviewer, which turns out to be the opposite of what we first assumed, and by stating the evidence level behind each of these claims.

3.1 Platform robustness: five defects fixed

Across our first runs, *AutoRecLab* never reached a final summary on native Windows. The blockers were not in the LLM-generated experiment code but in the framework and its platform assumptions. Table 1 lists the five defects we fixed in our fork (P1–P5) and a sixth (I1) that we reported as an upstream issue. The defects are independent: each one only became visible after the previous one was removed, which is why four separate runs were needed before a single run could finish. Three of the five are genuinely Windows-specific: a relative log path that breaks process spawning, a non-POSIX `os.kill` signal, and the resulting queue deadlock. The original paper does not state the operating system on which its experiments were run, but the project ships a recommended Docker setup that we deliberately bypassed (Section 2); whether these three defects arise on that supported configuration remains unconfirmed.

The most consequential fix is P5. After a sub-experiment exceeds the timeout, the cleanup logic kills the worker subprocess and drains its output queue; the subsequent read loop then blocks

Table 1: Defects we discovered in *AutoRecLab* and the fixes we applied in our fork (P1–P5). Three are Windows-specific (W). The encoding defect (I1) is the one we did not patch ourselves; we reported it as an upstream issue, which was subsequently fixed in the project.

ID	Defect and fix	Status
P1	matplotlib missing from dependencies; all drafts crash on import. Added it.	fixed
P2	Relative log path breaks multiprocessing spawn (W); workers cannot find out/debug.log. Made it absolute.	fixed
P3	IndexError when every node is buggy and the “best good node” list is empty. Added a fallback.	fixed
P4	os.kill(pid, SIGINT) raises PermissionError on Windows (W) at timeout. Skipped the POSIX path.	fixed
P5	Output-read loop deadlocks on an empty queue with a dead producer after a timeout (W). Added a bounded get.	fixed
I1	UnicodeEncodeError when an LLM reply contains non-ASCII characters on the Windows console.	reported; fixed upstream

forever on `queue.get()` because no producer will ever send the expected end-of-output marker. The unpatched system therefore hangs indefinitely after the first timeout. Replacing the blocking read with a bounded `get(timeout=2)` and an Empty handler lets the run continue to the next iteration, since a single slow sub-experiment would otherwise silently stall the entire agent.

3.2 Result quality appeared to track the LLM backend

In one observed run with gpt-5.4-mini and the subsequent runs with gpt-5.4, the larger model produced usable LensKit code earlier and more consistently. With gpt-5.4-mini, 6 of 13 debugging iterations failed on hallucinated LensKit imports and constructors (for example `lenskit.algorithms.knn`, `Popular`, `Pop`, and `from_interactions_df`), in which the model conflated a newer LensKit API (version 1.x or later) with the installed version 0.14.4. The same prompt with gpt-5.4 produced correct imports and a correct split on the first draft. With a single run per model this is an observation, not a controlled comparison: it does not establish a causal or general model effect, and repeated runs would be required to determine whether the difference is systematic. We ran all subsequent runs with gpt-5.4. We note that model choice did *not* determine methodological correctness: Runs 4, 5 and 7 all used gpt-5.4, and only Run 4 configured ItemKNN correctly (Section 3.4).

Table 2 summarises all seven runs and how each successive patch moved the system closer to a complete run. Run 5 is the first paper-scale run in our study to complete all ten tree-search iterations and print a final summary; only its repetition, Run 7, repeated this.

3.3 What reproduces, and what does not

Run 5 is our primary result: with all patches and a batch-recommend hint, the agent produced complete metric outputs for all 9 sub-experiments. The evaluated node constructs ALS as `ImplicitMF` with `features=50`, `iterations=15`, `weight=40`, `method='cg'` and `use_ratings=False`, and Most-Popular with `Popular()`. Of these, only `iterations=15` departs from a LensKit 0.14.4 default (which is 20), since `features` has no default and must be supplied. The ItemKNN constructor is treated separately in Section 3.4. Table 3 reports the mean `nDCG@10` and `Precision@10` over the five seeds, with the coefficient of variation. Figure 2 visualises the seed sensitivity directly: the near-flat lines show that changing the data-split seed barely moves `nDCG@10`.

One of the original study’s findings reproduces, one reproduces in an unexpected way, and one does not survive inspection.

Seed effects are small: this reproduces. Across the five tested split seeds the coefficient of variation of `nDCG@10` is between 1.3% and 2.8% for ALS and at most 5.7% for Most-Popular. Two qualifications belong with that number. The limited number of seeds does not establish how often the ordering of the two validly configured algorithms would change across a broader set of splits; and because the generated code leaves the model itself unseeded, these coefficients bound the split-seed effect from above rather than isolating it (Section 4). Within these runs the seed is nonetheless clearly a minor source of variation compared with the choice of algorithm. The one large CV (19.6% for Last.FM ItemKNN) divides by a near-zero mean and reflects the misconfiguration discussed below rather than real instability.

The paper’s main failure mode reproduces, and more literally than we first realised. The original study reports that its low ItemKNN values stem from a configuration unsuited to implicit feedback, which produced no error and therefore escaped its tree search. Our agent made the same class of mistake, in the same three ItemKNN slots, with the same invisibility to the reviewer (Section 3.4). We consider this the most substantive part of our reproduction.

The algorithm ranking does not reproduce as a scientific finding. The ordering `ALS > Pop > ItemKNN` is visible in our numbers, but it is produced by the misconfiguration: once ItemKNN is configured as LensKit 0.14.4 documents for implicit feedback, it overtakes Most-Popular on all three datasets and overtakes ALS on MovieLens. We therefore withdraw this finding rather than report it. Consequently the paired comparisons in Table 4 are informative only for ALS versus Most-Popular; the two comparisons involving ItemKNN measure the misconfiguration.

3.4 The ItemKNN sub-experiments are invalid, and why that matters

The original study attributes its near-zero ItemKNN values to a configuration unsuited to implicit feedback that produced no error and therefore escaped its tree search. We initially read our own near-zero values as a reproduction of that *numerical pattern*. Inspecting the evaluated node’s code shows that we reproduced the *configuration error* itself.

Both evaluated nodes construct the recommender as `ItemItem` with `nnbrs=20`, `min_nbrs=1`, `min_sim=1.0e-6` and `center=False`.

Table 2: Overview of our seven runs. Each row adds a patch or a setup change relative to the previous one; Run 7 repeats Run 5 under identical settings. We separate five states that the single label “valid” previously conflated: whether the framework terminated regularly, whether metric files were written, whether all expected algorithm–dataset combinations are present, whether we inspected the input data and the generated method at all, and how many of those combinations that inspection found methodologically sound. The last two are deliberately separate, because a sub-experiment can produce a complete and internally consistent set of numbers from an unsuitable configuration. “Combinations” counts algorithm $\times$ dataset pairs ($3 \times 3 = 9$ for the paper-scale prompt, 1 for the reduced prompt); each is averaged over 5 seeds (3 for the reduced prompt). The final column states what the run contributes to this paper.

Run	Model	Setup change	TS iters	Regular finish	Metrics present	Comb. complete	Data & method inspected	Method. validated	Used here	Runtime
1	gpt-5.4	original code, no patches	4/10 (hung)	no	no	0/9	partly	0/9	no	15 min
2	gpt-5.4-mini	+ P1, P2	10/10	no*	yes	9/9	yes	0/9*	no	19 min
3	gpt-5.4	+ P3	0/10 (exec. timeout)	no	no	0/9	partly	0/9	no	61 min
4	gpt-5.4	+ P4	7/10 (hung)	no	yes	9/9	yes	6/9†	partly†	124 min
5	gpt-5.4	+ P5, correct data file, batch hint	10/10	yes	yes	9/9	yes	6/9‡	yes	178 min
6	gpt-5.4	reduced prompt (1 $\times$ 1 $\times$ 3)	10/10	yes	yes	1/1	yes	1/1	yes	7 min
7	gpt-5.4	repeat of Run 5	10/10	yes	yes	9/9	yes	6/9‡	yes	62 min

API cost was ~\$0.30 (Runs 1, 3, 6), ~\$0.50 (Run 2) and ~\$0.5–1 (Runs 4, 5, 7); these are estimates, not metered figures.

*Run 2 wrote a complete set of 45 metric rows twice, but the values are degenerate: ALS and ItemKNN return bit-identical nDCG@10 in all 15 dataset–seed cells, and both are exactly 0.0 on Amazon Video Games. The reviewer marked every node buggy, and the run then crashed before a summary because the “good node” list was empty (defect P3).

†Run 4 hung before a summary, and its Amazon input was a defective file: the file has no header row, so the third column read as a rating is in fact a timestamp and the rating > 3 filter dropped none of its 231,780 rows. Its Amazon results therefore rest on corrupted input. Its MovieLens and Last.FM data are identical to Run 5’s, and on those two datasets the agent configured ItemKNN correctly for implicit feedback; we use Run 4 only as an internal control for that configuration (Section 3.4).

‡In Runs 5 and 7 the three ItemKNN combinations are technically completed but scientifically invalid: the generated code leaves ItemKNN in explicit-feedback mode, so every recommendation receives the identical score (Section 3.4). The six ALS and Most-Popular combinations are valid.

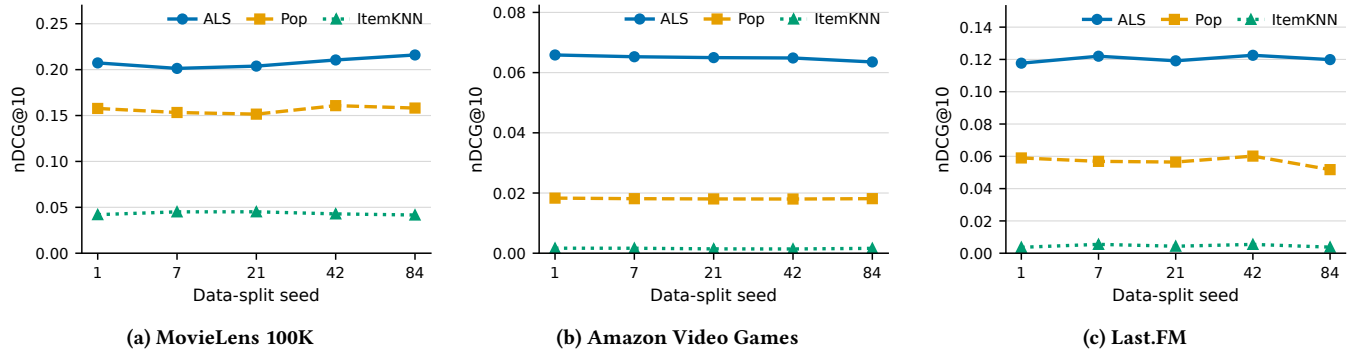


Figure 2: nDCG@10 across the five data-split seeds for each algorithm and dataset (Run 5, tree-search iteration 9). The lines are nearly flat, which is the seed-sensitivity result: changing the data-split seed barely moves nDCG@10. The vertical position of the ItemKNN line should not be read as a statement about ItemKNN: that configuration was left in explicit-feedback mode and assigns every recommendation the same score (Section 3.4).

In LensKit 0.14.4 the `ItemItem` constructor takes a feedback argument that defaults to ‘explicit’, and that default fixes two further settings: `aggregate=‘weighted-average’` and `use_ratings=True`. The generated code overrides only `center`. The training frame that reaches `fit` contains just user and item columns, so LensKit enters every interaction with an implicit value of 1.0. The weighted-average aggregate then computes, for each candidate item, $\sum_j (1 \cdot s_j) / \sum_j |s_j|$ over its rated neighbours, which is exactly 1.0 for every item that has at least one rated neighbour. The scores carry no information, and the resulting top- N list is decided entirely by tie-breaking.

LensKit does warn about this class of mistake, but not about this instance of it. Its constructor emits a `ConfigWarning` when a weighted-average aggregate is combined with `use_ratings=False`; here `use_ratings` is `True`, so the guard does not fire. The misconfiguration is silent at construction time and produces no error at run time, which is precisely why nothing in the tree search could catch it.

The evaluation around the algorithm is sound, which is what makes the failure hard to see. The generated code builds a per-user candidate map as the set difference between all items and the user’s training items, so items the user has already interacted

Table 3: Run 5 results (tree-search iteration 9, reviewer score 72/100; archived under artifacts/run5-selected-node/): mean nDCG@10 and Precision@10 over five data-split seeds (1, 7, 21, 42, 84). CV is the coefficient of variation of nDCG@10 across seeds. Values are reproduced verbatim from the node’s per-seed result file; we verified all nine means and CVs against it. The ItemKNN rows are reported for completeness only and are not valid measurements of ItemKNN: the algorithm was left in explicit-feedback mode, so every recommendation received the identical score (Section 3.4). Best *validly configured* algorithm per dataset in bold.

Dataset	Algorithm	nDCG@10	CV	P@10
MovieLens 100K	ALS	0.208	2.8%	0.151
	ItemKNN [†]	0.043	3.8%	0.035
	Pop	0.156	2.4%	0.120
Amazon Video Games	ALS	0.065	1.3%	0.020
	ItemKNN [†]	0.0016	7.7%	0.0005
	Pop	0.018	0.6%	0.006
Last.FM	ALS	0.120	1.7%	0.078
	ItemKNN [†]	0.005	19.6%	0.004
	Pop	0.057	5.7%	0.037

[†]Misconfigured; not a valid measurement of ItemKNN.

Table 4: Exploratory paired comparisons on five seed-matched splits ($n = 5$), per-seed nDCG@10, Run 5 (tree-search iteration 9). With five pairs these p -values are directional indicators, not calibrated significance levels. The lower block is listed separately because that ItemKNN configuration is invalid (Section 3.4); those rows quantify the misconfiguration, not an algorithm difference.

Dataset	Comparison	Δ nDCG@10	p
ML100K	ALS > Pop	+0.052	7.4e−6
Amazon	ALS > Pop	+0.047	2.2e−8
Last.FM	ALS > Pop	+0.063	2.2e−6
<i>Not valid as algorithm comparisons, because ItemKNN was misconfigured:</i>			
ML100K	ALS vs. ItemKNN [†]	+0.164	8.9e−7
ML100K	Pop vs. ItemKNN [†]	+0.113	1.1e−6
Amazon	ALS vs. ItemKNN [†]	+0.063	7.6e−9
Amazon	Pop vs. ItemKNN [†]	+0.017	7.4e−11
Last.FM	ALS vs. ItemKNN [†]	+0.116	3.4e−9
Last.FM	Pop vs. ItemKNN [†]	+0.052	2.3e−6

with are excluded from the candidate list. Re-executing the pipeline (Section 4) confirms this on a probe of evaluation users, and finds no overlap between the training and test interaction pairs. Both checks hold in all 44 re-executed cells: the 33 cells of the node itself plus the 11 counterfactual ItemKNN cells of Table 5. The metric code is likewise correct (Section 4). What is wrong is only the ranking that is fed into it.

We confirmed this by re-executing the evaluated node’s pipeline. In every ItemKNN run on MovieLens 100K and Last.FM, all recommendations receive the identical score: 9,380 and 10,900 recommendations, each set with exactly *one* distinct score and zero standard

Table 5: Effect of the ItemKNN misconfiguration. “As evaluated” is the configuration of the Run 5 node; “implicit” is feedback=’implicit’ as documented for LensKit 0.14.4. Values are nDCG@10 obtained by re-executing the node’s pipeline: means over the same five seeds for MovieLens 100K and Last.FM, seed 1 only for Amazon Video Games. The factors are computed from the unrounded values and therefore do not follow exactly from the rounded entries above. ALS and Most-Popular are shown as reference points.

Configuration	ML 100K	Amazon	Last.FM
ItemKNN, as evaluated	0.043	0.0014	0.005
ItemKNN, implicit	0.252	0.058	0.113
Factor	5.8×	41.4×	24.8×
Most-Popular	0.156	0.018	0.057
ALS	0.206	0.064	0.122
Corrected ranking	KNN > ALS > Pop		ALS > KNN > Pop

deviation. Because the identity follows algebraically from unary input, it holds for Amazon Video Games as well, and re-execution confirms it there too (633,830 recommendations, one distinct score).

The reported values then track candidate-set size rather than anything about the algorithm, as a signal-free ranking must: MovieLens 100K retains 1,008 items after filtering and yields 0.043, Last.FM 3,646 items and 0.005, Amazon Video Games 19,020 items and 0.0016. A larger candidate set makes it correspondingly less likely that an arbitrary top-10 happens to contain a held-out item.

The tie-breaking is arbitrary enough to be observable. On MovieLens 100K and Last.FM the reported ItemKNN values return bit-exactly when we re-execute the node, because item identifiers on those datasets are integers and the candidate sets, built as a Python set difference, therefore iterate in a fixed order. Amazon Video Games identifies items by string ASINs, whose set iteration order depends on Python’s per-process string hash seed, and the reported Amazon ItemKNN value does *not* return: we obtain nDCG@10 of 0.0014 against the 0.0017 reported for that same seed (Table 3 averages the five seeds to 0.0016). When every candidate carries the same score, the recommendation list is decided by an incidental property of the runtime, and the resulting metric is not reproducible across processes at all. The correct setting, which LensKit documents, is feedback=’implicit’ (equivalent to center=False, aggregate=’sum’ and use_ratings=False). Setting center=False alone, as the agent did, is not sufficient; our earlier reading of that flag as evidence of a correct implicit setup was mistaken.

Table 5 quantifies what this costs. Under the documented implicit configuration, ItemKNN’s nDCG@10 rises by a factor of 5.8 on MovieLens 100K, 41.4 on Amazon Video Games and 24.8 on Last.FM. It moves above Most-Popular on all three datasets and above ALS on MovieLens, so the reported ordering ALS > Pop > ItemKNN survives on none of the three once the algorithm is configured correctly: it is an artefact of the configuration.

Run 4 corroborates this independently. Its generated code tries ItemItem(nnbrs=20, feedback=’implicit’) first and therefore configured the algorithm correctly. On MovieLens 100K and Last.FM,

where Run 4 used exactly the same input data as Run 5, it obtained five-seed mean nDCG@10 of 0.256 and 0.116 (against 0.206 for its own ALS on MovieLens), matching our re-measurement of the corrected configuration to within 4% from an entirely separate code path. What Table 2 previously recorded as run-to-run variation was in fact the difference between a correct and an incorrect configuration.

Two consequences follow. First, only 6 of the 9 sub-experiments in Runs 5 and 7 are methodologically valid; the three ItemKNN combinations are technically completed but scientifically invalid. Second, and more interesting for an assessment of the tool: the agent’s reviewer *did* identify the problem. Requirement 13 of Run 5’s review states that “ItemKNN is not explicitly configured in an implicit-feedback mode as required/supported in LensKit 0.14.4”. The node was scored 72/100 and marked unsatisfactory partly for this reason, and the tree search then reported it anyway through the best-node fallback, with no indication in the final summary that a substantive methodological objection was outstanding. The failure is not that the reviewer missed the error; it is that nothing downstream of the reviewer acts on it.

Table 6 places our runs next to the original baseline. Both of our paper-scale runs produced complete metric outputs for all nine combinations, where the original left 12 of 27 sub-experiments invalid. Once methodological validity is required rather than completeness, however, our advantage largely disappears: we reach 6 of 9 valid combinations per run, and the three that fail are the same ItemKNN combinations that failed in the original. The tooling defects we fixed were real blockers, but removing them improved *completion*, not methodological correctness.

These rates should not be read as a controlled reliability comparison. The original study completed three of four started paper-scale runs; in our final patched phase, both of the two started paper-scale runs completed. The two figures refer to different software states: our two runs began only after five patches, a prompt extension, a model change, and a corrected input file, and the earlier development and failure runs are not counted in our denominator. Run 5 also differs from the original in several respects at once: our patches, a newer backbone model (gpt-5 $\rightarrow$ gpt-5.4), a slightly extended prompt, and a corrected Amazon data file (Table 2). No single factor can therefore be credited with the difference.

To probe run-to-run variation, we repeated Run 5 as Run 7 under identical settings, using the same prompt, configuration, and gpt-5.4 backend (Table 2). The repetition again produced complete metric outputs for all 9 combinations and again carried the ItemKNN misconfiguration in exactly the same three. The seed-sensitivity result is stable, with ALS CVs of 1.0–4.8% (Run 5: 1.3–2.8%), and ALS nDCG@10 deviates from Run 5 by at most 7% relative (0.202 vs. 0.208 on MovieLens, 0.061 vs. 0.065 on Amazon, 0.116 vs. 0.120 on Last.FM); the agent even generated the same five seeds (1, 7, 21, 42, 84) again. Note that Run 7 repeats the *pipeline*, not Run 5’s generated code: the agent wrote different code that happens to share the same defect. One consequence is worth stating precisely, because it is easy to over-read. Run 7 wrote its own nDCG and Precision implementation, independent of the one Run 5 generated, and the two agree closely; that is evidence about the *metric computation*, not about the run. A second invocation of the agent cannot corroborate the first run’s numbers, only its behaviour. The

Table 6: Our patched paper-scale runs (Runs 5 and 7) versus the original proof-of-concept [4]. The two completion rates refer to different software states and should not be interpreted as a controlled reliability comparison.

Metric	Paper	Runs 5 and 7 (patched)
Started runs completed	3/4	2/2 (different software state)
Complete metric outputs	not reported	18/18
Methodologically valid	15/27 (56%)	12/18 (67%)
Runtime	2–9.5 h	1–3 h
API cost	\$1–5	~\$0.5–1
Seed effect	small	small (CV 1.0–4.8% for ALS)
ALS vs. Most-Popular	ALS > Pop	ALS > Pop
Dominant failure mode	ItemKNN misconfigured	ItemKNN misconfigured

agent’s behaviour varied noticeably between the two runs: in Run 7 the reported implementation appeared already in tree-search iteration 2, the remaining eight iterations all failed on two recurring bugs (five on the `ImplicitMF()` constructor error that dominated Run 5, three on a schema mismatch in the generated Amazon loader), and the run finished in 62 minutes, roughly a third of Run 5’s runtime.

3.5 Small, scoped tasks run fast and cheap

To test the agent on an easier task, we ran the reduced prompt (ALS on MovieLens 100K, three seeds, default hyper-parameters). The agent finished in 7 minutes for about \$0.30 and reached a score of 83.33/100. The result reproduced the seed-stability finding (CV $\approx$ 1% over three seeds). Notably, this run is also the only one of ours in which we found no methodological defect: it asks for a single algorithm that needs no feedback-mode configuration, so the failure mode of Section 3.4 cannot arise, and its `ImplicitMF` call leaves every hyper-parameter that has a default at that default, as the prompt asked. This rests on inspection of the node’s code and result file (L1, Table 8); we did not re-execute this run, and its ALS is unseeded in the same way as Run 5’s (Section 4). That is the more precise reading of this result: a narrower prompt does not just give the agent fewer opportunities to fail at the level of execution, it also removes the configuration decisions on which it silently fails. *AutoRecLab* is therefore not unusable; it is most trustworthy when the task is small and unambiguous.

3.6 Discussion: the reviewer rejects, but nothing acts on the rejection

Across every run, the reviewer never marked a node as satisfactory. Our best paper-scale node scored 72/100 (in Run 5 and again, with the identical score, in Run 7), and our best small-task node scored 83.33/100, yet all were flagged unsatisfactory. Our first reading of this was that the reviewer is miscalibrated and rejects its own valid work. Going through its objections one by one against the artefacts (Table 7) does not support that reading.

Table 7: Every objection the reviewer raised against the two selected nodes (seven unmet requirements in Run 5 and five in Run 7), numbered as in each run’s requirement list and checked against the artefacts, plus one requirement that Run 7’s reviewer marked as fulfilled. The full per-requirement feedback is archived as `reviewer_score.txt` in each node directory. Two of the objections are substantive; we had previously treated all of them as formal.

Objection	Verdict	Basis
R5-13 ItemKNN not implicit	upheld, subst.	Re-execution: every score is 1.0; correcting it raises $nDCG@10$ 5.8–24.8× (Sec. 3.4)
R5-21 Significance on 5 seeds	upheld, subst.	p -values to $7e-11$ from five pairs
R5-12, R7-1 Hyperparameters off defaults	upheld, but mis-exemplified	Prompt asked for standard hyperparameters. The reviewer’s examples <code>features=50</code> and <code>nnbrs=20</code> are required arguments without defaults in LensKit 0.14.4; the actual deviations are <code>iterations=15</code> (default 20) and <code>center=False</code> , and in Run 7 additionally <code>reg=0.01</code> (default 0.1)
R5-18/19, R7-16 Output not in tidy form	upheld, formal	Wide format; values unaffected
R5-23, R7-17 Missing input validation	upheld, formal	No file, schema or empty-data checks
R7-9 Evaluated users not reported per dataset and seed	upheld, formal	Counts absent from the result files; values unaffected
R7-15 Seed sensitivity not summarised per combination	upheld, formal	The node reports CV but no range (<code>cv_summary.csv</code>) and does not identify the most seed-sensitive combinations; values unaffected
R5-1 LensKit 0.14.4 compat. “not ensured”	not upheld	Code ran end to end on 0.14.4 without API errors
R7-10 Implicit-feedback training “fulfilled”	missed	Marked fulfilled although Run 7’s ItemKNN carries the identical defect to Run 5’s

Three things need separating here. *Miscalibration* is the smallest of them: only one objection (Run 5’s requirement 1) is unfounded, and four more are formal points that do not affect any reported value (Table 7). A fifth, that the nodes did not leave the hyperparameters at their defaults, is correct as a verdict even though the reviewer picked the wrong examples for it (Table 7); it explains none of the errors we found. *Substantively correct criticism* is the largest: the reviewer identified both the ItemKNN misconfiguration and the over-reading of five-seed significance tests. These are the two most serious methodological problems in the node, and precisely the two we had to correct in this paper. Charging the reviewer with over-strictness was our error, not its own. *The missing fallback* is the actual defect: without our P3 fix, an empty “good node” list crashes the run outright, so a user can do everything right and receive no output at all; with the fix, the best node is reported

through a path that surfaces neither the score nor the outstanding objections (Section 3.4).

A fourth observation cuts against reading the reviewer as dependable. Run 7’s node carries the ItemKNN defect in a character-for-character identical constructor, yet its requirement 10, “train each algorithm ... using implicit-feedback data in the format expected by LensKit 0.14.4”, is marked fulfilled, and no requirement in that run’s feedback mentions the feedback mode at all. The same reviewer, on the same prompt, with the same defect in front of it, caught the problem once and missed it once; its substantive objections are therefore worth acting on when they appear, but their absence is not evidence that a node is sound.

That is the reliability gap we would now name: not that the reviewer is too strict, but that its verdict has no consequence. A system that reports the affected numbers anyway, without flagging them, is more dangerous than one that simply fails; the cheapest improvement would be for the final summary to carry the score and the unmet requirements alongside the results.

3.7 Verdict: proof-of-concept or practical tool?

Returning to the question in our title, the evidence supports a nuanced answer. As a turnkey *tool*, *AutoRecLab* is not yet there, which is to be expected from an Alpha release: it does not run out of the box on Windows, and its output quality appears to track an expensive backend model. Most seriously, it reported a methodologically invalid configuration as a result in both paper-scale runs, once after its own reviewer had identified the problem and once after that reviewer had certified the very requirement the code violated (Section 3.6). As a *proof-of-concept*, it delivers a good deal. Once the platform blockers are removed, each of our two \$0.5–1 paper-scale runs produced complete, internally consistent metric outputs for all nine combinations, recovered the original study’s seed-sensitivity result, and did so in one to three hours. On a small, clearly scoped task our single run was fast and cheap, and we found no methodological defect in it on inspection. Reproducing the original’s ItemKNN misconfiguration can be read two ways: as a successful replication of its dominant failure mode, or as evidence that the failure is a property of the approach rather than of one study. We therefore read *AutoRecLab* as a convincing proof-of-concept whose remaining gaps are engineering and tuning problems. One qualification matters: the gap that matters most is not platform robustness or model choice but the absence of any mechanism that stops an unmet methodological requirement from being reported as a result. For a first Alpha release, this is a solid foundation with one sharp edge.

4 VALIDATION AND EVIDENCE

Reproduction studies are only as good as the checks behind them, and “we verified the results” can mean anything from reading a summary to reimplementing a pipeline. We therefore state, for each part of our work, how thoroughly it was actually checked. We use five levels: **L0** figures taken from the agent’s report without inspecting code, logs or result files; **L1** code, logs or result files visually inspected; **L2** the generated code re-executed separately and the new results compared with the reported ones; **L3** central steps checked with our own independent logic; **L4** results reproduced

with an independent implementation or external reference. We follow a matching convention in the text, and the wording is chosen so that the level is readable from the verb. “*AutoRecLab* reported” would signal L0; no claim in this paper rests on it. “The generated artefacts contain” and “we confirmed by inspecting” signal L1. “We confirmed this by re-executing” and “values are reproduced verbatim from” signal L2. “We reimplemented” and “we recomputed ... with our own code” are reserved for L3, where a step was checked with logic we wrote ourselves rather than with the agent’s. Nothing in this paper claims L4, so no phrasing is reserved for it.

Before stating levels we fix what completeness means here. The prompt implies 3 datasets $\times$ 3 algorithms = 9 aggregated sub-experiments, 9×5 seeds = 45 algorithm–dataset–seed runs, and 45×2 metrics $\times$ 3 cut-offs = 270 individual metric values. Both Runs 5 and 7 meet this in full: all 45 rows and all 270 values are present, every value is unique to its combination and finite, each combination carries exactly five distinct seeds, and no combination is missing or duplicated.

Table 8 gives the levels we reached. One boundary of the L2 claim should be stated at the outset: our re-execution script carries the node’s pipeline functions as a transcription rather than importing the archived file, so that the loop around them can be driven per dataset and per seed. We verified that transcription by comparing abstract syntax trees: every pipeline function used, and the algorithm constructors, are identical to the node’s. The archive records that comparison.

Three further things are worth calling out. The completeness and preprocessing checks are genuinely independent (L3): we reimplemented 5-core filtering and recomputed the interaction counts for all three datasets from the raw files, matching the recorded 54,413/533,133/52,551 interactions exactly, and we ran the completeness checks above from our own code rather than the agent’s. The metric computation is also L3: we reimplemented Precision@ k and nDCG@ k as a plain per-user loop, deliberately written differently from the node’s vectorised version, and compared the two on every re-executed cell. They agree to 1.1×10^{-16} across all 33 cells, so the node’s metric code is correct for all nine combinations. Held against the *reported* per-seed file rather than against the re-run, our implementation returns the published values exactly for five of the nine combinations: Most-Popular on all three datasets and ItemKNN on the two with integer item identifiers; the four that differ are precisely the non-deterministic ones, the three ALS combinations and Amazon ItemKNN. And the re-execution (L2) revealed a limitation we would otherwise have missed: Most-Popular reproduces bit-exactly everywhere and ItemKNN does on the two datasets with integer item identifiers, but ALS does not, because the generated code passes no random state to ImplicitMF and its factor matrices are therefore initialised unseeded. Individual ALS values shift by up to 7.6% between executions, although the five-seed means agree to within 1.7%. The prompt asked for reproducible execution with explicit seeds; the generated code seeds the data split but not the model, and neither the reviewer nor our own earlier inspection caught this.

This bears directly on the one finding we report as reproduced. Because the model is unseeded, the coefficients of variation of Section 3.4 measure the split seed and the initialisation together, so they are an upper bound on the split-seed effect and not a clean

Table 8: Evidence level reached per part of our study. L0 unchecked, L1 inspected, L2 re-executed, L3 independently checked, L4 independently reimplemented.

Part	Level	Basis
Platform defects and patches	L2	Each defect localised, patched, behaviour observed after the fix
Preprocessing and 5-core counts	L3	Reimplemented; counts recomputed for all three datasets
Completeness (9/45/270)	L3	Checked systematically for Runs 5 and 7
Metric computation	L3	Independent reimplementation; agrees with the node’s on all 33 cells, and returns the published values exactly for 5 of 9 combinations
Algorithm configuration	L3	Constructor instantiated and its resolved settings read back from LensKit; checked against the LensKit source
Central Run 5 numbers	L2	Node pipeline re-executed; exact for Most-Popular everywhere and for ItemKNN on MovieLens 100K and Last.FM, approximate for ALS, not reproducible for Amazon ItemKNN
Data splitting and candidate sets	L2–L3	Split re-executed: no train/test overlap and no seen items among the candidates in any of the 44 cells; resulting training and test sets not otherwise independently validated
Run 7 repetition	L1	Repeats the pipeline, not Run 5’s code; does not raise Run 5’s level
Run 6 (small task)	L1	Selected node’s code and result file inspected; not re-executed and not covered by our two scripts
Reviewer calibration	L2	Objections mapped to artefacts; requirement 13 confirmed by re-execution

estimate of it. On Last.FM the two are of comparable size: the five-seed CV of ALS is 1.7%, while re-running the *same* seed shifts individual values by up to 7.6%. The finding itself survives, since the total variation from both sources together is small, which is what “seed effects are small” asserts; the attribution to the split alone does not, and separating the two would require re-running each seed several times with different model initialisations.

Two boundaries should be stated plainly. Nothing in this study reaches L4: we did not reproduce any result with an independent recommender implementation or against an external reference. And the L2 and L3 checks cover a sample rather than the full grid: we re-executed all five seeds on MovieLens 100K and Last.FM but only seed 1 on Amazon Video Games, whose scale makes a full re-run expensive. The claims in this paper are scoped accordingly.

5 LIMITATIONS

Our study has nine limitations. First, we report only two complete paper-scale runs (Run 5 and its repetition, Run 7); although both produced complete metric outputs and closely agreeing metrics, two runs remain a small sample for characterising run-to-run variation, and the reported API costs and runtimes are estimates rather than metered figures. Second, our runs cover only five data-split seeds, as specified in the original prompt. This is enough to show that the seed is a minor source of variation within these runs, but it is a small sample for the paired t -tests, whose very small p -values should be read as directional rather than precise. The limited number of seeds does not establish how often a different ranking would occur across a broader set of splits. Third, we did not tune hyper-parameters and deliberately requested defaults, so our accuracy numbers reflect out-of-the-box behaviour, not best-case performance. Fourth, we tested only the OpenAI model family (gpt-5.4 and gpt-5.4-mini), with a single run per model, so the apparent model effect is an observation rather than a controlled result. Fifth, we evaluated only the native Windows environment and the initial Alpha release. We did not run the project’s recommended Docker setup, so we cannot confirm whether our three Windows-specific defects also arise on that supported configuration. We also did not test the actively developed develop branch, on which the issue we reported (I1) was subsequently fixed and on which some of our other defects may already be addressed. All of our work was carried out on the Alpha release, while a more recent version of *AutoRecLab* is under active development, which likely resolves several of the defects we describe and presumably delivers better results; our findings should therefore be read as a snapshot of the early Alpha release, not as an assessment of the current system. Sixth, our re-execution covers all five seeds on MovieLens 100K and Last.FM but only seed 1 on Amazon Video Games, so the corrected-ItemKNN comparison rests on a single split for that dataset (Section 4). Seventh, our re-execution showed that ALS results are not exactly reproducible from the generated code, because it leaves ImplicitMF unseeded; the ALS values we report are therefore one draw from a distribution, with per-seed deviations of up to 7.6% and five-seed means stable to within 1.7%. Eighth, three of the nine sub-experiments per run rest on an ItemKNN configuration that is invalid for implicit feedback; we report their values because they document a reproducible failure mode, but they carry no information about ItemKNN’s accuracy, and any conclusion that would require them (in particular a three-way algorithm ranking) is outside what our data support. Finally, we did not measure energy consumption or CO₂, and we leave a Docker/Linux comparison to future work; the Unicode defect (I1) that we reported has since been fixed upstream.

6 CODE AND ARTIFACT AVAILABILITY

All code, patches, prompts, configurations, logs, and results are available in our fork [9] at <https://github.com/robin-nitschke/AutoRecLabTeam5>, which is based on the initial Alpha release of *AutoRecLab* [3]. This paper refers to the repository state at commit 46e219e (29 July 2026). The repository distinguishes between files that document development failures and files that support the numbers reported here. The two nodes whose output this paper reports are archived separately, each with its code, node identifier or

tree-search iteration, reviewer score and complete per-requirement feedback, full agent log, prompt, configuration, dependency pins, per-seed result file, aggregated result table, and statistical output:

- `artifacts/run5-selected-node/`: Run 5, tree-search iteration 9, reviewer score 72/100.
- `artifacts/run7-selected-node/`: Run 7, node `dac1170c438449f5a248475720dd3267`, iteration 2, reviewer score 72/100.

Each directory carries a README.md explaining every file and a NODE.json with machine-readable provenance including code hashes. The canonical result files behind our tables are `per_seed_results.csv` in these two directories. Two scripts reproduce the checks reported in Section 4: `scripts/validate_results.py` re-derives the completeness and preprocessing checks without importing LensKit or any *AutoRecLab* code, and additionally resolves the archived ItemKNN constructor against LensKit; `scripts/rerun_selected_node.py` re-executes the evaluated node’s pipeline and produces the corrected-ItemKNN comparison of Table 5. Their recorded output, including the syntax-tree comparison that verifies the re-execution against the archived node, is in `artifacts/VALIDATION_REPORT.md` and `artifacts/RERUN_REPORT.md`. ARTIFACTS.md lists the artefacts of all seven runs and what each is good for; REPRODUCING.md gives versions, installation, and exact commands.

ACKNOWLEDGEMENTS

During the writing of this paper, we used AI tools including Claude (Anthropic) for drafting and refining text, analysing experimental results, and reviewing code patches. We manually inspected the selected nodes, their execution logs, their per-requirement reviewer feedback and their result files, and verified that the reported seeds, algorithms and aggregated values are present in those artefacts. We recomputed the aggregates of Table 3 and the completeness of both result files with our own script, resolved the ItemKNN configuration against the LensKit source, and re-executed the Run 5 node’s pipeline as described in Section 4. We did not independently reimplement the complete evaluation pipeline, and no result here was reproduced with an independent recommender implementation. The experimental runs, code modifications, and conclusions are our own work. Our usage aligns with the AI usage guidelines of the Intelligent Systems Group at the University of Siegen [1].

REFERENCES

- [1] Joeran Beel. 2024. Our Use of AI-Tools for Writing Research Papers. Intelligent Systems Group, Blog, University of Siegen. <https://isg.beel.org/blog/2024/08/19/our-use-of-ai-tools-for-writing-research-papers/>
- [2] Joeran Beel and Moritz Baumgart. 2026. Intelligent Systems Seminar. *Universität Siegen* (2026).
- [3] Joeran Beel, Bela Gipp, Tobias Vente, Moritz Baumgart, Philipp Meister, and Sinan Pourazari. 2026. AutoRecLab Alpha: Source Code. <https://github.com/ISG-Siegen/AutoRecLab>.
- [4] Joeran Beel, Bela Gipp, Tobias Vente, Moritz Baumgart, Philipp Meister, and Sinan Pourazari. 2026. AutoRecSys or AutoRecLab? A Call to Build, Evaluate, and Govern Autonomous Recommender-Systems Research Labs, and a First Proof-of-Concept. *ACM Transactions on Recommender Systems* (2026). Preprint.
- [5] Michael D. Ekstrand. 2020. LensKit for Python: Next-Generation Software for Recommender Systems Experiments. In *Proceedings of the 29th ACM International Conference on Information and Knowledge Management (CIKM)*. Association for Computing Machinery, 2999–3006. doi:10.1145/3340531.3412778

- [6] Zhengyao Jiang, Dominik Schmidt, Dhruv Srikanth, Dixing Xu, Ian Kaplan, Deniss Jacenko, and Yuxiang Wu. 2025. AIDE: AI-Driven Exploration in the Space of Code. <https://github.com/WecoAI/aideml>. arXiv:2502.13138
- [7] Jiaqi Liu, Shi Qiu, Mairui Li, Bingzhou Li, Haonian Ji, Siwei Han, Xinyu Ye, Peng Xia, Zihan Dong, Meng Chen, Congyu Zhang, Letian Zhang, Guiming Chen, Haoqin Tu, Xinyu Yang, Lu Feng, Xujiang Zhao, Haifeng Chen, Jiawei Zhou, Xiao Wang, Weitong Zhang, Hongtu Zhu, Yun Li, Jieru Mei, Hongliang Fei, Jiaheng Zhang, Linjie Li, Linjun Zhang, Yuyin Zhou, Sheng Wang, Caiming Xiong, James Zou, Zeyu Zheng, Cihang Xie, Mingyu Ding, and Huaxiu Yao. 2026. AutoResearchClaw: Self-Reinforcing Autonomous Research with Human-AI Collaboration. <https://github.com/aiming-lab/AutoResearchClaw>. arXiv:2605.20025
- [8] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. 2024. The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery. <https://github.com/SakanaAI/AI-Scientist>. arXiv:2408.06292
- [9] Robin Nitschke and Simon Kontaxakis. 2026. AutoRecLab Team 5 Fork: Code, Patches, Prompts, and Results. <https://github.com/robin-nitschke/AutoRecLabTeam5>.
- [10] Lukas Wegmeth, Tobias Vente, Lennart Purucker, and Joeran Beel. 2023. The Effect of Random Seeds for Data Splitting on Recommendation Accuracy. In *Proceedings of the 3rd Workshop on the Perspectives on the Evaluation of Recommender Systems (PERSPECTIVES 2023), co-located with RecSys 2023 (CEUR Workshop Proceedings, Vol. 3476)*. CEUR-WS.org. <https://ceur-ws.org/Vol-3476/paper4.pdf>
- [11] Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune, and David Ha. 2025. The AI Scientist-v2: Workshop-Level Automated Scientific Discovery via Agentic Tree Search. <https://github.com/SakanaAI/AI-Scientist-v2>. arXiv:2504.08066